

Architecture de Transition : Table vs Kanban

Unicité de la logique métier dans IncidentFlow

Concept clé (Single Source of Truth) : Les vues Tableau Classique et Kanban ne sont que deux interfaces graphiques différentes. L'intelligence métier (les règles de transition) est centralisée. Si un administrateur modifie le workflow, les deux vues s'adaptent instantanément sans aucune duplication de code logique.

1. L'Interception Visuelle : Mode Kanban

Dans le Kanban, l'utilisateur a la liberté de glisser une carte n'importe où. Le frontend intercepte cette action (le *Drop*) et vérifie si le mouvement correspond à une flèche existante dans le Workflow actif.

Fichier : frontend/src/KanbanView.jsx

```
const handleDrop = useCallback((e, targetStatus) => { e.preventDefault(); // 1.
  Récupération du statut actuel et du statut cible const currentStatus =
  (draggedIncident.status || '').toLowerCase(); // 2. Validation par rapport aux
  règles du workflow actif if (activeWorkflow && activeWorkflow.transitions) { const
  allowedTransitions = activeWorkflow.transitions.filter( t => (t.fromState ||
  '').toLowerCase() === currentStatus ); const matchedRule = allowedTransitions.find(
  t => (t.toState || '').toLowerCase() === targetStatus.toLowerCase() ); // 3. Rejet
  si le mouvement est illégal (pas de flèche dans le workflow) if (!matchedRule) {
  const validNextStates = allowedTransitions.map(t => t.toState).join(', ');
  setPendingTransitionModal({ isForbidden: true, forbiddenReason: `Depuis
  "${currentStatus}", les seuls états autorisés sont : ${validNextStates}` }); return;
  } } });
```

Si la transition est invalide, la carte retourne à sa position initiale et une alerte affiche dynamiquement les seuls états autorisés.

2. L'Anticipation Visuelle : Mode Détails (Tableau)

Dans la vue classique, la logique est la même, mais présentée de manière préventive. Le frontend filtre les transitions et n'affiche sous forme de boutons **que** les états d'arrivée autorisés. L'utilisateur ne peut donc physiquement pas déclencher d'erreur.

3. L'Ultime Barrière de Sécurité : Le Backend (Java)

Le frontend n'est jamais digne de confiance absolue. Quelle que soit l'interface utilisée, une requête API est envoyée au backend `performTransition()`. C'est ici qu'intervient la validation définitive. Si la transition demandée n'existe pas dans la base de données, le backend lève une exception (HTTP 400).

Fichier : `backend/src/.../WorkflowService.java`

```
public WorkflowTransition validateTransitionForIncident(Incident incident, String
fromState, String toState, User user, String comment) { // 1. Récupération du
workflow global validé par le DFS Workflow workflow = incident.getWorkflow(); if
(workflow == null) { workflow = workflowRepository.findFirstByActiveTrue()
.orElseThrow(() -> new InvalidTransitionException( "Aucun workflow global actif
configuré dans le système.")); } // 2. Recherche stricte de la flèche de transition
dans la DB WorkflowTransition match = workflow.getTransitions().stream() .filter(t
-> t.getFromState().equalsIgnoreCase(fromState) &&
t.getToState().equalsIgnoreCase(toState)) .findFirst() .orElseThrow(() -> new
InvalidTransitionException( String.format("Transition de '%s' vers '%s' non
autorisée.", fromState, toState) )); return match; }
```

Conclusion pour la Soutenance

Le système a été conçu selon les meilleures pratiques de sécurité : **une validation UI (Frontend)** pour le confort utilisateur (alerte Kanban et masquage des boutons) et **une validation stricte (Backend)** pour garantir l'intégrité de la base de données. L'algorithme *DFS* protège le Workflow lors de sa création, et cette architecture protège chaque ticket lors de son cycle de vie.